

4

Testing conditions

This chapter demonstrates the various PHP control structures that determine how a script will proceed.

Seeking truth

Providing alternatives

Switching branches

Matching expressions

Performing loops

Looping while true

Breaking from loops

Summary

Seeking truth



The **if** keyword is used to perform the basic conditional test that evaluates a given expression for a Boolean value of **TRUE** or **FALSE**. Statements within braces following the evaluation will only be executed when the expression is found to be **TRUE**. When the expression is found to be **FALSE**, the PHP parser simply moves on to the next part of the script.

The syntax of an **if** conditional test statement looks like this:

```
if ( test-expression ) { statement-to-execute-when-true ; }
```

There may be multiple statements to be executed when the test is **TRUE** but each statement must be terminated by a semi-colon:

```
if ( test-expression )  
{  
    statement-to-execute-when-true ;  
    statement-to-execute-when-true ;  
    statement-to-execute-when-true ;  
}
```



The **if** construct is one of the most important features in PHP, and in most other programming languages.

Multiple expressions can be tested using the logical **&&** or **and** operator to ensure that following statements will only be executed when both expressions are found to be **TRUE**, like this:

```
if ( ( test-expression ) && ( test-expression ) )  
{
```

```

    statement-to-execute-when-true ;

    statement-to-execute-when-true ;

    statement-to-execute-when-true ;

}

```

Alternatively, **if** conditional test statements can be nested to test multiple expressions so that statements will only be executed when all tests are found to be **TRUE**, like this:

```

if ( test-expression )
{
    if ( test-expression )
    {
        statement-to-execute-when-true ;

        statement-to-execute-when-true ;

        statement-to-execute-when-true ;

    }
}

```



When the code to be executed is just a single statement, the braces may optionally be omitted, but you may prefer to always include the braces for consistency.

Multiple **if** conditional test statements can be made to provide alternatives for the script to follow.

1 Between PHP tags, begin a script with a statement that initializes an integer variable

```

<?php
$num = 8 ;

```



if.php

2 Next, add statements that output a result when a conditional test of numerical size is found to be

```

TRUE

if ( $num > 5 ) { echo "$num exceeds 5<br>" ; }

```

```
if ( $num <= 5 ) { echo "$num is below 6<br>" ; }
```

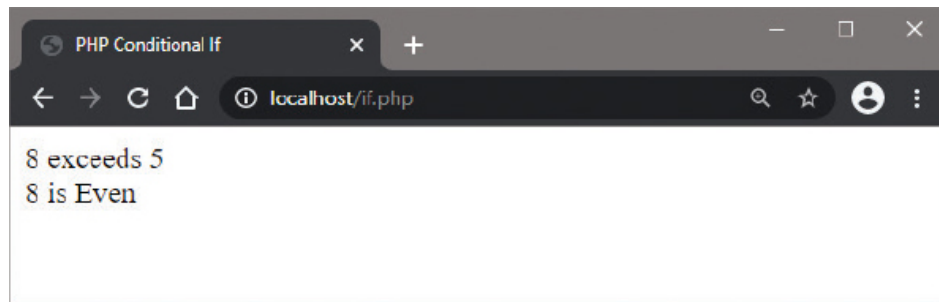
3 Now, add statements that output a result when a conditional test of parity is found to be **TRUE**

```
if ( $num % 2 == 0 ) { echo "$num is Even<br>" ; }
```

```
if ( $num % 2 != 0 ) { echo "$num is Odd<br>" ; }
```

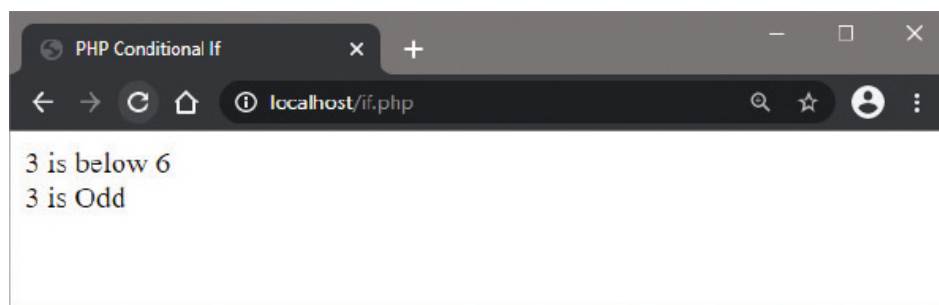
```
?>
```

4 Save the document in your web server's **/htdocs** directory as **if.php** then open the page via HTTP to see the results from successful tests get displayed



5 Edit the integer variable then save the modified script and refresh the browser to see the results from other successful tests get displayed

```
$num = 3 ;
```



When the tested condition is **FALSE**, the PHP parser ignores its statement block and simply moves on.

Providing alternatives



It is often preferable to extend an **if** statement by appending an **else** statement, specifying statements within braces to be executed when the expressions evaluated by the **if** statement are found to be **FALSE**, with this syntax:

```
if ( test-expression )
{
    statement-to-execute-when-true ;
    statement-to-execute-when-true ;
    statement-to-execute-when-true ;
}
else
{
    statement-to-execute-when-false ;
    statement-to-execute-when-false ;
    statement-to-execute-when-false ;
}
```

Alternative expressions may also be evaluated in a conditional test by adding an **elseif** statement, specifying statements within braces to be executed when the alternative expression is found to be **TRUE**. An **else** statement can be appended here too, to specify statements within braces to be executed when the expressions evaluated by the **if** and **elseif** statements are found to be **FALSE**, with this syntax:

```
if ( test-expression )
{
    statement-to-execute-when-true ;
    statement-to-execute-when-true ;
    statement-to-execute-when-true ;
}
elseif ( test-expression )
```

```

{
    statement-to-execute-when-true ;
    statement-to-execute-when-true ;
    statement-to-execute-when-true ;
}
else
{
    statement-to-execute-when-false ;
    statement-to-execute-when-false ;
    statement-to-execute-when-false ;
}

```



The **elseif** keyword can also be written as two separate words: **else if**.

This is a fundamental programming technique that offers the script different directions in which to proceed, depending on the result of the evaluations, and is known as “conditional branching”.



You may come across some PHP scripts that use a : colon character rather than curly brackets. This is not recommended as **if** and **else if** will only be considered exactly the same when using curly brackets.

1 Between PHP tags, begin a script with a statement that outputs a result only when a tested expression is **TRUE**

```
<?php
```

```
if ( 4 > 2 ) { echo 'Yes, 4 is greater than 2 <br>' ; }
```



ifelse.php

2 Next, add a statement that outputs a result only when two tested expressions are both found to be **TRUE**

```
if ( ( 4 > 2 ) && ( 8 > 4 ) )
{ echo '4 is greater than 2 AND 8 is greater than 4<br>' ; }
```

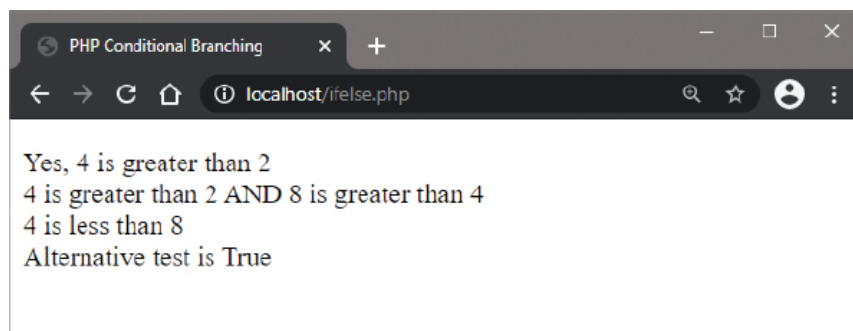
3 Now, add a statement that outputs a result when a tested expression is found to be **TRUE** or **FALSE**

```
if ( 4 > 8 )
{ echo '4 is greater than 8 <br>' ; }
else
{ echo '4 is less than 8 <br>' ; }
```

4 Finally, add a statement that outputs a result when either of two tested expressions are found to be **TRUE** or when both tested expressions are found to be **FALSE**

```
if ( 4 > 8 )
{ echo 'This test is True </p>' ; }
elseif ( 8 > 4 )
{ echo 'Alternative test is True </p>' ; }
else
{ echo 'Both tests are False </p>' ; }
?>
```

5 Save the document in your web server's **/htdocs** directory as **else.php** then open the page via HTTP to see the results of conditional branching get displayed



You can nest conditional **if** tests (one inside another) so that statements in the inner block only get executed when both tests are found to be **TRUE**.

Switching branches



Conditional branching performed by multiple **if-elseif-else** statements can often be performed more efficiently by a **switch** statement when the test expression just evaluates one condition.

The **switch** statement works in an unusual way. It takes a given value as its parameter argument then seeks to match that value from a number of **case** statements. Code to be executed when a match is found is included in each **case** statement.

It is important to end each **case** statement with a **break** keyword statement, so the **switch** statement will then exit when a match is found without seeking further matches – unless that is the deliberate requirement.

Optionally, the list of **case** statements can be followed by a single final **default** statement to specify code to be executed in the event that no matches are found within any of the **case** statements. So the syntax of a switch statement typically looks like this:

```
switch ( test-value )  
{  
    case match-value : statements-to-execute-when-matched ; break ;  
    case match-value : statements-to-execute-when-matched ; break ;  
    case match-value : statements-to-execute-when-matched ; break ;  
    default : statements-to-execute-when-no-match-found ;  
}
```

A **switch** statement can have many **case** statements, but no two of its **case** statements can attempt to match the same value.



It's always a good idea to include a **default** statement – even if it's only to output an error message when the **switch** statement fails.

Where a number of match-values are to each execute the same statements, only the final **case** statement need include the statements to be executed and the **break** statement to exit the **switch** statement block.

For example, to output the same message for match-values of 0, 1 and 2:

```
switch ( $number )
{
    case 0 :
    case 1 :
    case 2 : echo 'Less than 3' ; break ;
    case 3 : echo 'Exactly 3' ; break ;
    default : echo 'Greater than 3 or less than zero' ;
}
```



A **default** statement need not appear at the end of the **switch** block, but it is logical to place it there where it needs no **break** statement.

1 Between PHP tags, begin a script with statements that initialize two variables with an integer and character value

```
<?php
```

```
$number = 2 ; $letter = 'B' ;
```



switch.php

2 Next, add a **switch** statement to match the integer value

```
switch ( $number )
```

```
{
```

```
    case 1 : echo 'Number is One<br>' ; break ;
```

```

case 2 : echo 'Number is Two<br>' ; break ;
case 3 : echo 'Number is Three<br>' ; break ;
default : echo 'Number is Unrecognized<br>' ;
}

```

Now, add a **switch** statement to match the character value

```

switch ( $letter )
{
case 'A' : echo 'Letter is A<br>' ; break ;
case 'B' : echo 'Letter is B<br>' ; break ;
case 'C' : echo 'Letter is C<br>' ; break ;
default : echo 'Letter is Unrecognized<br>' ;
}

```

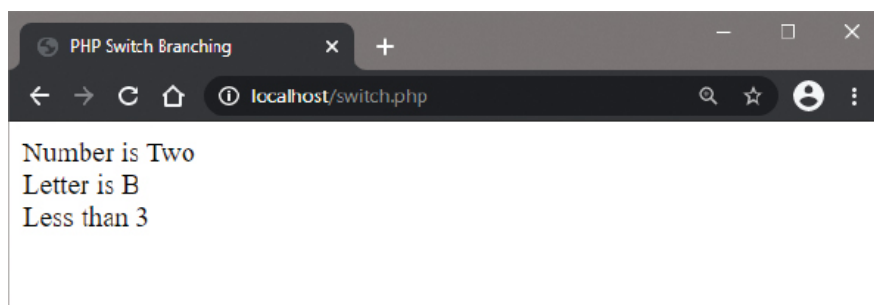
Finally, add a **switch** statement to match against multiple integer values

```

switch ( $number )
{
case 0 : case 1 : case 2 : echo 'Less than 3<br>' ; break ;
default : echo '3 or more, or less than zero' ;
}
?>

```

Save the document in your web server's **/htdocs** directory as **switch.php** then open the page via HTTP to see the results of conditional branching get displayed



In **switch** statements, the **case** keyword, match-value and colon character are regarded as a unique “label”.

Matching expressions



Conditional branching can also be performed by **match** statements. These are an alternative to the **switch** statements, described [here](#), and may often be a better choice.

Unlike a **switch** statement block, a **match** statement is a single expression that returns a value. It takes a given value as its parameter argument then seeks to match that value from a comma-separated list of alternatives enclosed within curly brackets. The value to be returned when a match is found is then specified after a **=>** fat arrow operator.

It is important to end each **match** statement with a ; semi-colon after the closing curly bracket to terminate the statement.

A **match** expression does not require any **break** statements so its code is significantly shorter than an equivalent **switch** block. The syntax of a **match** statement typically looks like this:

```
$result = match ( test-value )
{
    match-value => return-value ,
    match-value => return-value ,
    match-value => return-value ,
    default => return-value ,
};
```



The **match** statement is a great new feature introduced in PHP 8.

Where a number of match-values are to each return the same value, they can be combined into a comma-separated single list. For example, to output the same message for match-values of 0, 1 and 2:

```

echo match ( $number )
{
    0 , 1 , 2 => 'Less than 3' ,
    3 => 'Exactly 3' ,
    default => 'Greater than 3 or less than zero' ,
};

```

It is useful to recognize that a **match** statement compares values for identity, like using the **===** operator, whereas a **switch** statement compares values for equality, like using the **==** operator.

The steps in the example opposite use **match** statements to replicate the output from the previous **switch** statement example.

- 1 Between PHP tags, begin a script with statements that initialize two variables with an integer and character value

```

<?php
$number = 2 ; $letter = 'B' ;

```



match.php

- 2 Next, add a **match** statement to match the integer value

```

echo match ( $number )
{
    1 => 'Number is One<br>' ,
    2 => 'Number is Two<br>' ,
    3 => 'Number is Three<br>' ,
    default => 'Number is Unrecognized<br>' ,
};

```

- 3 Now, add a **match** statement to match the character value

```

echo match ( $letter )
{
    'A' => 'Letter is A<br>' ,
    'B' => 'Letter is B<br>' ,

```

```

    'C' => 'Letter is C<br>',
    default => 'Letter is Unrecognized<br>',
};

```

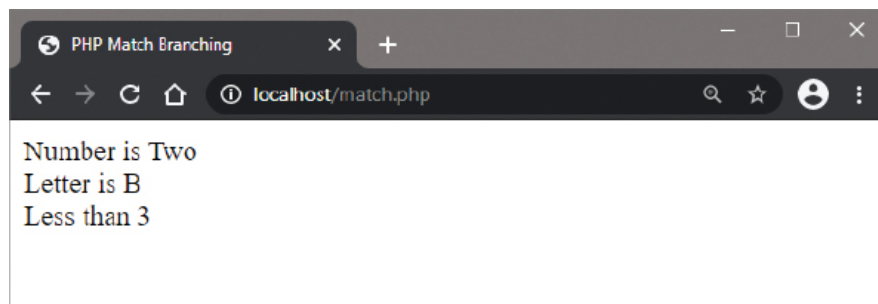
4 Finally, add a **match** statement to match against multiple integer values

```

echo match ( $number )
{
    0 , 1 , 2 => 'Less than 3<br>',
    default => '3 or more, or less than zero' ,
};
?>

```

5 Save the document in your web server's **/htdocs** directory as **match.php** then open the page via HTTP to see the results of conditional branching get displayed



Compare the code in this example against the **switch** statements in the previous example to see the simplicity advantage of **match** statements.

Performing loops



A loop is a piece of code in a script that automatically repeats. One complete execution of all statements within a loop is called an “iteration”, or a “pass”. The length of a loop is controlled by a conditional test made within the loop. While the tested expression is found to be **TRUE** the loop will continue – until the tested expression is found to be **FALSE** at which point the loop ends.

In PHP there are four types of loop structure:

- **for** loop – performs a specified number of iterations.
- **while** loop – performs iterations only while a test made at the start of each iteration evaluates as **TRUE**.
- **do while** loop – performs iterations only while a test made at the end of each iteration evaluates as **TRUE**.
- **foreach** loop – performs iterations to traverse each element of an array, as introduced [here](#).

Perhaps the most interesting loop structure is the **for** loop, which typically has this syntax:

```
for ( initializer ; test-expression ; updater )  
{  
    statement-to-execute-when-true ;  
    statement-to-execute-when-true ;  
    statement-to-execute-when-true ;  
}
```

The initializer is used to set a starting value for a counter of the number of iterations made by the loop. An integer variable is used for this purpose and is traditionally named “i”.

Upon each iteration of the loop, the test-expression is evaluated and that iteration will only continue while this expression is **TRUE**. When the test-expression becomes **FALSE**, the loop ends immediately without executing the statements again. With every iteration the counter is updated then the statements executed.



The updater may be an incrementer, or it may be a decrementer in a loop that counts down rather than counting up.

Loops can be nested, one within another, to allow complete execution of all iterations of an inner nested loop on each iteration of the outer loop.

1 Between PHP tags, create a script with a **for** loop to output the loop counter value on each of three iterations

```
<?php
for ( $i = 1 ; $i < 4 ; $i++ )
{
    echo "<dt>Outer loop iteration $i" ;

    # Nested loop to be inserted here.
}
?>
```



forloop.php

2 Save the document in your web server's **/htdocs** directory as **forloop.php** then open the page via HTTP to see the loop counter values get displayed

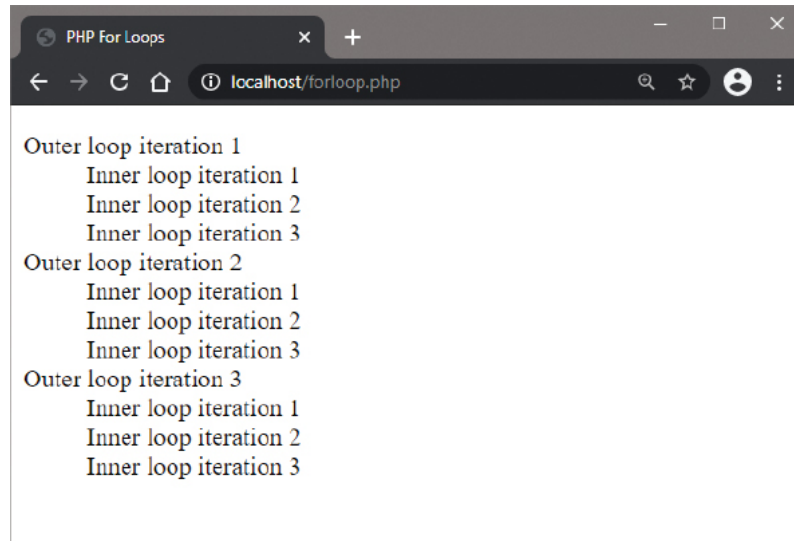
3 Now, insert a nested **for** loop to output its loop counter value in a list on each of three iterations

```
for ( $j = 1 ; $j < 4 ; $j++ )
{
    echo "<dd>Inner loop iteration $j" ;
```



```
}
```

4 Save the document once more then open the page via HTTP to see both loop counter values get displayed



```
Outer loop iteration 1
    Inner loop iteration 1
    Inner loop iteration 2
    Inner loop iteration 3
Outer loop iteration 2
    Inner loop iteration 1
    Inner loop iteration 2
    Inner loop iteration 3
Outer loop iteration 3
    Inner loop iteration 1
    Inner loop iteration 2
    Inner loop iteration 3
```



A **for** loop counter can also count down – by decrementing the counter value on each iteration using **\$i--** in place of **\$i++**.

Looping while true



A **while** loop is an alternative to the **for** loop described in the previous example. The **while** loop also requires an initializer, test-expression, and an updater, but these are not neatly listed within a single pair of parentheses as they are in a **for** loop. Instead, the initializer must appear before the start of the loop block, and the test-expression must appear within parentheses after the **while** keyword, followed by braces containing both an updater and the statements to be executed on each iteration.

initializer

```
while ( test-expression )  
{  
    statement/s-to-be-executed ;  
    updater ;  
}
```

A **do while** loop provides a subtle variation on the syntax above by placing the **do** keyword before the loop block and moving the **while** statement to after the loop block:

initializer

```
do {  
    statement/s-to-be-executed ;  
    updater ;  
}  
while ( test-expression ) ;
```



Ensure that the test expression in a loop structure will at some point become **FALSE** to avoid creating an infinite loop that never ends.

Both **while** loops and **do while** loops will proceed to make iterations until the test-expression is found to be **FALSE**, at which point the loop will exit. It is therefore essential that the loop body contains code that will, at some point, change the result of the test-expression evaluation – otherwise an infinite loop is created, which will lock the system. The significant difference between a **while** loop and a **do while** loop is that a **while** loop will not make a single iteration if the test-expression is **FALSE** on its first evaluation. In contrast, a **do while** loop will always make at least one iteration because its statements are executed before the evaluation is made. If this is desirable, a **do while** loop is obviously the best choice, otherwise the choice between a **for** loop and a **while** loop is largely a matter of personal preference. One rule of thumb suggests that a **for** loop is best used to perform a specific number of iterations, whereas a **while** loop is best used to iterate until a certain condition is met. Loops are the perfect partner to use with arrays as each iteration can effortlessly read or write into successive array elements.

1 Between PHP tags, begin a script with a statement to create an array

```
<?php
```

```
$numbers = array( 10 , 20 , 30 ) ;
```



whileloop.php

2 Next, add a **while** loop to output the value of each array element

```
echo '<dt>While Loop : ' ;
```

```
$i = 0 ;
```

```
while ( $i < 3 )
```

```
{
```

```
    echo "<dd>Element $i = $numbers[$i] " ;
```

```
    $i++ ;
```

```
}
```

3 Now, add a **do while** loop to output the value of each array element

```
echo '<dt>Do While Loop : ' ;
```

```
$i = 0 ;
```

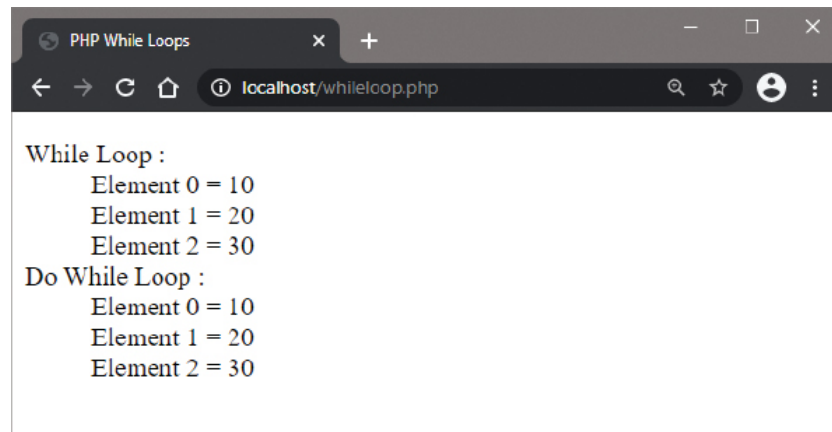
```
do
```

```
{
```

```
    echo "<dd>Element $i = $numbers[$i] " ;
```

```
    $i++;  
}  
while ( $i < 3 );  
?>
```

4 Save the document in your web server's **/htdocs** directory as **whileloop.php** then open the page via HTTP to see the array element values get identically displayed



Change the value in this test expression to zero – so **while (\$i = 0)** in both loops – to see only the first iteration of the **do while** loop get executed.

Breaking from loops



The **break** keyword can be used to prematurely terminate a loop when a specified condition is met. The **break** statement is situated inside the loop statement block and is preceded by a test expression. When the test returns **TRUE**, the loop ends immediately and the program proceeds on to the next task. For example, in a nested inner loop the program proceeds to execute the next iteration of the outer loop.

1 Between PHP tags, begin a script with a **for** loop that performs three iterations

```
<?php
for ( $i = 1 ; $i < 4 ; $i++ )
{
    # Nested loop to be inserted here (Step 2).
}
```



breakcontinue.php

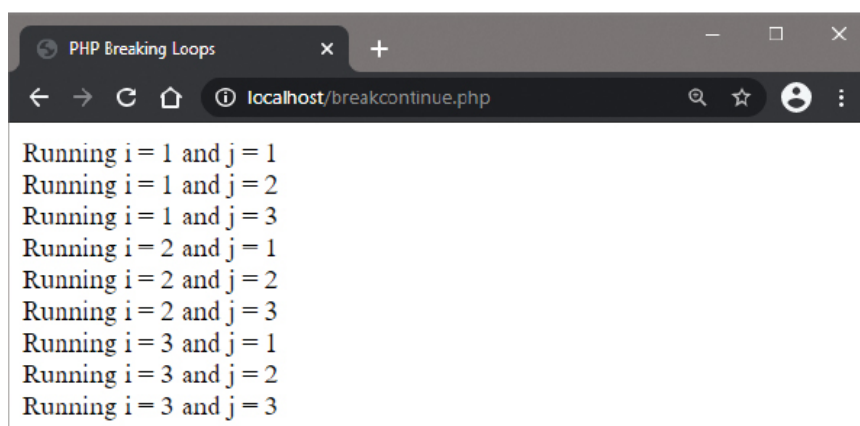
2 Next, insert a nested **for** loop that also performs three iterations

```
for ( $j = 1 ; $j < 4 ; $j++ )
{
    # Statements to be inserted here (Steps 5 & 6).
}
?>
```

3 Now, insert a statement that outputs the value of both loop counters on each iteration

```
echo "Running i = $i and j = $j <br>" ;
```

4 Save the document in your web server's **/htdocs** directory as **breakcontinue.php** then open the page via HTTP to see both loop counter values get displayed



```

Running i = 1 and j = 1
Running i = 1 and j = 2
Running i = 1 and j = 3
Running i = 2 and j = 1
Running i = 2 and j = 2
Running i = 2 and j = 3
Running i = 3 and j = 1
Running i = 3 and j = 2
Running i = 3 and j = 3

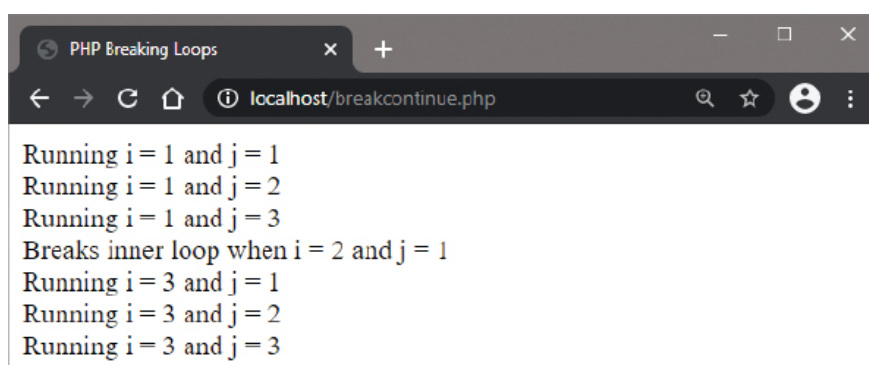
```

Next, insert this **break** statement at the very beginning of the inner loop block, to break out of the inner loop – then save the document and view the change

```

if ( $i == 2 && $j == 1 )
{
    echo "Breaks inner loop when i = $i and j = $j <br>" ;
    break ;
}

```



```

Running i = 1 and j = 1
Running i = 1 and j = 2
Running i = 1 and j = 3
Breaks inner loop when i = 2 and j = 1
Running i = 3 and j = 1
Running i = 3 and j = 2
Running i = 3 and j = 3

```

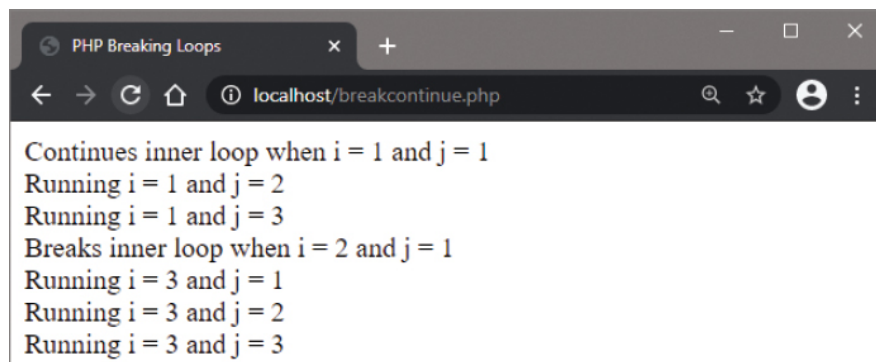


Here, the **break** statement halts all three iterations of the inner loop when the outer loop tries to run it the second time.

The **continue** keyword can be used to skip a single iteration of a loop when a specified condition is met. The **continue** statement is situated inside the loop statement block and is preceded by a test expression. When the test returns **TRUE**, that single iteration ends.

Now, insert this **continue** statement at the beginning of the inner loop block, to skip the first iteration of the inner loop – then save the document and view the change

```
if ( $i == 1 && $j == 1 )  
{  
    echo "Continues inner loop when i = $i and j = $j <br>" ;  
    continue ;  
}
```



```
Continues inner loop when i = 1 and j = 1  
Running i = 1 and j = 2  
Running i = 1 and j = 3  
Breaks inner loop when i = 2 and j = 1  
Running i = 3 and j = 1  
Running i = 3 and j = 2  
Running i = 3 and j = 3
```



Here, the **continue** statement just skips the first iteration of the inner loop when the outer loop tries to run it for the first time.

Summary

- The **if** keyword performs the basic conditional test that evaluates an expression for a Boolean value of **TRUE** or **FALSE**.
- Statements within braces following the evaluation will only be executed when the expression is found **TRUE**.
- Multiple expressions can be tested using the logical **&&** operator, or by nesting multiple **if** conditional tests.
- The **else** keyword specifies statements to be executed when the expression evaluated by the **if** statement is found **FALSE**.
- The **elseif** keyword is used to append an alternative expression to be evaluated and statements to be executed when **TRUE**.
- Multiple **if-elseif-else** statements can often be performed more efficiently by a **switch** statement.
- Each **case** statement specifies a value to be matched in a **switch** statement block and must end with a **break** keyword.
- A **default** statement specifies code to be executed in the event that no matches are found within any **case** statements.
- A **match** statement is a single expression that returns a value when it finds a match to its specified parameter.
- A loop is a piece of code in a script that automatically repeats until a test expression becomes **FALSE**.
- The PHP loop structures are **for**, **while**, **do while** and **foreach**.
- A **for** loop structure specifies an initializer, a test expression and an updater that increments (or decrements) a counter.
- An updater must always be included in the statement block of a **while** loop and a **do while** loop.
- Unlike a **while** loop, a **do while** loop will always execute its statements at least once.
- The **break** keyword can be used to prematurely terminate a loop when a specified condition is met.
- The **continue** keyword can be used to skip a single iteration of a loop when a specified condition is met.